LAB S11 / MEXI BANCO

Una verdad, varias copias

Bitacora de ejecucion: PostgreSQL como verdad del dinero, Redis para la lectura del saldo y MongoDB para la bitacora de avisos.

munoz_victoria / Sistemas Distribuidos, FI-UNAM, 2027-1	trabajo-s11 sobre 06-poliglota-base	6 de noviembre de 2026

Resultado del laboratorio

Implemente cache-aside en las tres replicas de cuenta. El GET cacheable solo muestra el saldo; los cargos y abonos siguen leyendo y modificando PostgreSQL. Los avisos se guardan como documentos por canal en MongoDB, con identificador idempotente e indice para la consulta de los diez mas recientes.

La guia permite presentar evidencia como texto o captura. Esta bitacora usa extractos textuales de las respuestas, los logs y las consolas reales del entorno local; los tiempos de Spring se muestran en UTC.

Datos reutilizados

El volumen de PostgreSQL ya tenia las CLABEs 002180000000000001, ...002 y ...003 de practicas previas. Se conservaron sus saldos existentes en lugar de borrar el volumen: al iniciar, Ana (Cuenta Origen) tenia $598, Beto (Cuenta Destino) $902 y Carla $300. La guia contempla cuentas ya existentes.

Entorno

```
docker compose up --build -d
docker compose ps  # servicios: healthy
docker compose exec -T redis redis-cli PING  -> PONG
docker compose exec -T mongo mongosh notificacion --eval
  'db.avisos.countDocuments()'               -> 0
```

El Compose levanto las tres replicas de cuenta, Postgres, Redis, MongoDB, RabbitMQ, discovery, gateway y servicios de apoyo. Redis y Mongo respondieron antes de medir.

1. Punto de partida - diez consultas, diez lecturas

Para la linea base configure CACHE_TIPO=none con la variable prevista en application.yml, prepare la transferencia inicial de $200 de Ana a Beto y consulte diez veces la cuenta de Ana. El primer POST de transferencia respondio 201 / COMPLETADA; despues Ana quedo en $398 y Beto en $1,102.

```
POST /api/transferencias  HTTP 201
{"id":15,"monto":200,"estado":"COMPLETADA",
 "saldoOrigenTrasCargo":398.00,"eventoId":"2e4f2aaf-..."}
```

Las diez respuestas tuvieron saldo 398 y un leidaEn distinto. Este extracto recoge las horas de lectura de Postgres:

```
GET 1  05:23:24.540588345Z
GET 2  05:23:24.639687640Z
GET 3  05:23:24.689184696Z
GET 4  05:23:24.726190467Z
GET 5  05:23:24.755964052Z
GET 6  05:23:24.799171655Z
GET 7  05:23:24.829311019Z
GET 8  05:23:24.859786450Z
GET 9  05:23:24.885889534Z
GET 10 05:23:24.918831295Z
```

Conteo de log antes	Conteo despues	Diferencia	Redis
6 lecturas acumuladas	16 lecturas acumuladas	10 nuevas lecturas a Postgres	keyspace_hits=0; keyspace_misses=0

El delta a partir de los logs de las tres replicas fue exactamente diez. Redis no tuvo hits ni misses en esta medicion.

El conteo acumulado inicial incluia solicitudes de preparacion; compare el mismo log inmediatamente antes y despues de las diez consultas para aislar el delta.

2. La cache compartida

En ConfiguracionCache active la abstraccion de cache de Spring, limite cada entrada a 30 segundos, serialize CuentaVista como JSON y use transactionAware(). ConsultaDeSaldo usa @Cacheable con la CLABE como llave. CuentaService conserva su consulta interna sin cache para autorizar operaciones; cargar y abonar invalidan la llave con @CacheEvict despues del commit.

Dicho en mis palabras: el primer GET de una CLABE pregunta a Redis. Si falta, el metodo consulta Postgres, devuelve el saldo y Spring deja una copia temporal en Redis. Las tres instancias de cuenta apuntan al mismo servicio Redis, asi que una copia creada por una replica sirve a las otras dos. Cuando una transferencia cambia el saldo, se borra esa copia despues de confirmar el cambio; el siguiente GET la reconstruye desde Postgres.

Las tres instancias contestaron el mismo valor y la misma marca leidaEn:

```
GET 1 X-Instancia=9ca3998c70d6  leidaEn=05:25:05.366Z
GET 2 X-Instancia=d1ea296d53d2  leidaEn=05:25:05.366Z
GET 3 X-Instancia=bb3b21ed2868  leidaEn=05:25:05.366Z
GET 4 X-Instancia=9ca3998c70d6  leidaEn=05:25:05.366Z
GET 5 X-Instancia=d1ea296d53d2  leidaEn=05:25:05.366Z
GET 6 X-Instancia=bb3b21ed2868  leidaEn=05:25:05.366Z
Lecturas nuevas a Postgres en esos seis GET: 1
```

Redis guardo un JSON legible y la llave tenia 28 segundos restantes al tomar la evidencia:

```
GET cuentas::0021800000000000001
{"clabe":"002180000000000001","titular":"Cuenta Origen",
 "saldo":398.00,"leidaEn":"2026-11-07T05:25:05.366131843Z"}
TTL cuentas::0021800000000000001
(integer) 28
```

Invalidacion observada

La transferencia de $10 respondio 201 y dejo a Ana en $388. MONITOR mostro el GET usado para consultar la cuenta, luego DEL de las llaves de Ana y Beto. EXISTS para Ana dio 0. El siguiente GET leyo el saldo nuevo y obtuvo otro leidaEn.

```
DEL cuentas::0021800000000000001
DEL cuentas::0021800000000000002
EXISTS cuentas::0021800000000000001 -> 0
GET posterior: saldo=388.00, leidaEn=05:25:42.327Z
SET ... PX 30000
```

3. Mis medidas

Para los hit ratios use 100 consultas por escenario y CONFIG RESETSTAT. El hit ratio es hits / (hits + misses). En B intercale una transferencia de $1 despues de cada diez consultas: diez transferencias completadas en total.

Escenario	Consultas	Redis hits / misses	Hit ratio	Lecturas Postgres
A - 100 GET, sin cambios	100	99 / 1	99.0%	1
B - 100 GET + 10 transferencias	100 + 10	100 / 20	83.3%	20

El ratio baja en B porque cada transferencia invalida las copias de origen y destino. Al volver a consultar, hay que volver a leer una o ambas cuentas desde Postgres; esas lecturas son necesarias para rellenar la cache.

Latencia por corrida, 100 GET por corrida

Cache	Corrida	Promedio (ms)	Mediana (ms)	p95 (ms)	Postgres
Redis	1	11.73	11.15	15.66	0
Redis	2	11.51	10.84	16.52	0
Redis	3	12.00	10.85	20.37	1
Sin cache	1	26.38	15.78	29.27	100
Sin cache	2	16.44	15.39	23.24	100
Sin cache	3	15.74	13.91	27.06	100

La tercera corrida con Redis tuvo un miss observado y una lectura a Postgres; lo conservo en la tabla. En el resto de corridas cacheadas no hubo lecturas a Postgres durante las 100 solicitudes. Las corridas sin cache produjeron 100 lecturas cada una.

Interpretacion: en este loopback local, las medianas con cache quedaron entre 10.84 y 11.15 ms; sin cache, entre 13.91 y 15.78 ms. La primera media sin cache fue mas alta (26.38 ms), por lo que reporto cada corrida y no solo un promedio combinado. La mejora mas clara es capacidad: una tanda caliente de 100 GET evita casi todas las 100 lecturas repetidas a Postgres.

4. Avisos en MongoDB: tres formas

El consumidor de RabbitMQ ahora llama a bitacora.registrar(evento), que inserta solo en MongoDB. Cada aviso es un documento con campos comunes y los campos que necesita su canal. La coleccion empezo en cero avisos; el indice se creo al arrancar notificacion.

```
[avisos] indice clabe_fecha listo en avisos
```

Hice una transferencia de Ana a Beto por $200 y otra de Ana a Carla por $50. mongosh find().sort({fecha:1}) devolvio cuatro documentos; la proyeccion con telefono devolvio solo las dos filas SMS de Ana.

Canal y destinatario	Campos propios observados	Campos de dinero
SMS - Ana, CLABE ...001 (2 avisos)	telefono=+52 55 5555 0101; texto	monto Decimal128; saldo Decimal128
CORREO - Beto, CLABE ...002	correo=beto@correo.mx; asunto; cuerpo	monto Decimal128; saldo Decimal128
PUSH - Carla, CLABE ...003	token=fcm:demo-...003; titulo; cuerpo	monto Decimal128; saldo Decimal128

Ejemplo de documento devuelto por mongosh:

```
{
  _id: "94df6028-...:0021800000000000001",
  eventoId: "94df6028-ad2f-4632-a6a4-c7c668813dd6",
  transferenciaId: Long(27), clabe: "002180000000000001",
  tipo: "TRANSFERENCIA_ENVIADA",
  monto: Decimal128("200"), saldo: Decimal128("178.00"),
  fecha: ISODate("2026-11-07T05:35:00.048Z"),
  canal: "SMS", telefono: "+52 55 5555 0101", texto: "..."
}
```

El _id es eventoId + ':' + clabe. Un reintento del evento no crea un segundo aviso para la misma CLABE. El monto y el saldo se guardan como Decimal128, no como double.

5. Endpoint, indice y explain

Despues hice 12 transferencias de $1 de Ana a Beto. Las 12 respondieron HTTP 201. Quedaron 28 documentos en avisos: cuatro de las dos primeras transferencias y 24 de las doce pequeñas.

```
GET /api/notificaciones/002180000000000002
HTTP 200
ENDPOINT_CANTIDAD=10 CANALES=CORREO
Orden: del mas nuevo al mas viejo; los dos primeros eran
transferenciaId 40 y 39, monto=1, saldo Beto=1334 y 1333.
```

La consulta de indice devolvio _id_ y clabe_fecha con key {clabe:1, fecha:-1}. Compare los dos executionStats con 28 documentos en la coleccion:

Consulta	Plan observado	totalDocsExamined	nReturned
Con indice	LIMIT -> FETCH -> IXSCAN (clabe_fecha)	10	10
Sin indice, hint $natural	SORT -> COLLSCAN	28	10

```
db.avisos.find({clabe: "...002"})
  .sort({fecha:-1}).limit(10).explain("executionStats")
// IXSCAN keyPattern {clabe:1, fecha:-1}; examina 10

db.avisos.find({clabe: "...002"})
  .sort({fecha:-1}).limit(10).hint({$natural:1})
  .explain("executionStats")
// SORT + COLLSCAN; examina los 28
```

Con esta coleccion pequena, la diferencia es 10 frente a 28 documentos examinados. El indice sigue la consulta: iguala por CLABE y recorre fecha en descendente.

6. Evento duplicado y dato viejo

Evento duplicado

Publique dos veces el mismo evento fijo (transferenciaId 9001, eventoId 11111111-2222-4333-8444-555555555555, CLABEs inventadas ...091 y ...092) desde el API de RabbitMQ. Ambas publicaciones respondieron routed=true. Mongo termino con dos documentos para el evento, uno por CLABE, y el segundo intento registro ambos _id como ya existentes.

```
publish 1 -> HTTP 200 {"routed":true}
publish 2 -> HTTP 200 {"routed":true}
db.avisos.countDocuments({eventoId: "1111...5555"}) -> 2
[avisos] ...:002180000000000091 ya estaba: evento repetido
[avisos] ...:002180000000000092 ya estaba: evento repetido
[consumidor] ... 0 aviso(s) nuevo(s)
```

Dato viejo

Force una cache fresca de Ana en $116 (TTL 30), luego cambie directamente PostgreSQL a $5. El GET aun mostro $116 con TTL 28. Mientras esa copia estaba presente intente transferir $100. La respuesta fue HTTP 422, "Saldo insuficiente": la autorizacion leyo Postgres. La clave aun existia despues del rechazo. En la siguiente consulta la clave vieja ya habia expirado y el GET devolvio $5 con un leidaEn nuevo.

Evidencia	Valor observado
Copia antes del UPDATE	$116; leidaEn 05:38:20.997Z; TTL 30 s
Postgres actualizado fuera de la app	$5 (UPDATE 1)
GET mientras seguia la copia vieja	$116; TTL 28 s
Transferencia de $100	HTTP 422; saldo insuficiente en Postgres
Siguiente lectura	$5; TTL anterior -2; leidaEn 05:38:29.000Z

La marca leidaEn vieja y la nueva difieren aproximadamente 8.0 segundos en las muestras capturadas. Despues restaure el valor de Ana que habia antes de la prueba y elimine esa unica clave de cache para cerrar el ejercicio con datos de cuenta utilizables.

7. Redis caido y preguntas de cierre

Reto opcional

Implemente un CacheErrorHandler que registra los fallos de GET, SET, DEL y CLEAR. El GET que no puede contactar Redis cae a Postgres. El DEL despues del commit se registra sin convertir la transferencia confirmada en error.

```
Con Redis detenido:
GET /api/cuentas/...071 -> HTTP 200, saldo 100.00
POST /api/transferencias (...071 -> ...072, $1)
  -> HTTP 201, estado COMPLETADA
Postgres cuenta: ...071 = 99.00; ...072 = 1.00
Postgres ledger: ...071 = 99.00; ...072 = 1.00
Logs: [cache] GET / SET / DEL fallo; se uso Postgres
Redis reencendido: PING -> PONG
GET ...071 -> saldo 99.00; GET ...072 -> saldo 1.00
```

Para esta comprobacion cree dos cuentas temporales nuevas con deposito inicial y asiento de ledger propios. Asi los saldos y la suma de movimientos cuadraron en la prueba de fallo, sin mezclar las cuentas que ya existian en el volumen.

Preguntas de cierre

1. Mongo para avisos, no para saldo. Los avisos tienen formas variables por canal, se escriben una vez y se leen enteros por CLABE. Un documento expresa SMS, correo o push sin columnas vacias ni ALTER TABLE. El saldo autoriza cargos y debe mantener invariantes transaccionales; por eso su verdad permanece en PostgreSQL.

2. Medi 99.0% en A y 83.3% en B. Las transferencias de B borraban las copias de Ana y Beto, por lo que las consultas posteriores provocaron misses. Las medianas con cache fueron 10.84-11.15 ms; sin cache, 13.91-15.78 ms. La mejora local de latencia fue moderada y variable; el ahorro claro fue capacidad de Postgres: de 100 lecturas a 0 en dos corridas calientes, y 1 en la tercera.

3. Si el cargo autorizara con Redis, una copia de $116 podria autorizar un retiro de $100 aunque Postgres tuviera $5. El experimento exhibio el saldo viejo al cliente y rechazo el retiro con 422 porque la regla leyó la verdad en PostgreSQL.

4. Borro la copia en vez de escribir el nuevo saldo porque Redis es una copia reconstruible: el proximo GET trae el resultado confirmado desde Postgres. El borrado se difiere hasta despues del commit para que otra consulta no repueble Redis con el saldo viejo durante la transaccion y para no invalidar una operacion que despues se revierta.

Estado de entrega: se incluyen las carpetas completas cuenta y notificacion sin target, y esta bitacora. No cambie docker-compose.yml.